

RailEasy (Foundation) - Capstone Project for Certification

Contents

Purpose

1) About Us

2) Problem Statement

3) Roles

4) In-Scope Features

5) User Flows (Happy Path)

7) Architecture

8) Tech Stack

9) Data Model

10) API Outline

11) NFRs & SLAs

12) Testing

13) Engineering PoV

14) Delivery Plan

15) Sample Acceptance Criteria

16) Example Seed Data (SQL)

17) Simple Wireframe

18) Sample User Stories

19) Definition of Ready (DoR) & Done (DoD)

20) Evaluation Rubric (Aligned to Foundation Skills)

22) Stretch Goals (Optional, If Time Permits)

23) Getting Started (Suggested Commands)

Purpose

Build a train ticket booking web application where users can search trains between stations, choose a travel class, select seats, and confirm a booking. Admins manage trains and train schedules. The focus is on foundational full-stack skills - React/Angular, Spring Boot, REST, PostgreSQL, Git, and basic testing - aligned with the Foundation skill matrix.

1) About Us

At RailEasy, we believe train travel should be booked in minutes, not hours. This Foundation version delivers a clean train search, class selection, and booking experience with essential features only.

2) Problem Statement

Train travellers need a simple way to find trains between stations, view class availability, and confirm a ticket without navigating complex government portals.

Goal: Deliver a minimal yet complete train booking flow demonstrating full-stack fundamentals.

3) Roles

1. Passenger/User - Can search trains, view class availability, book and cancel tickets.
2. Admin - Can manage trains and schedules.

Note: Simple isAdmin boolean flag on the User entity.

Role Flag / Field Approach

- User entity: boolean isAdmin; (defaults to false on registration)
- Seed one admin user with isAdmin = true.
- Backend: check isAdmin before allowing train/schedule CRUD.
- Frontend: show Admin menu only when isAdmin is true in user context.

Backend Concept

- Backend: check isAdmin before allowing train/schedule CRUD.
- Frontend: show Admin menu only when isAdmin is true in user context.

Frontend Concept

- Backend: check isAdmin before allowing train/schedule CRUD.
 - Frontend: show Admin menu only when isAdmin is true in user context.
-

4) In-Scope Features

Authentication (Basic)

- Register, Login, Logout.
- Simple password hashing (BCrypt).

Train Search

- Search form: From Station, To Station, Date.
- Results: train name, number, departure, arrival, available classes with seat counts and fare.

Class & Seat Selection

- Select a travel class (SLEEPER / AC_3 / AC_2).
- Simple seat grid for selected class (8 rows x 8 seats = 64 per class).
- Seats shown as Available or Booked.
- Select 1-4 seats and confirm.

Booking & My Tickets

- Confirm booking -> PNR number generated (UUID first 8 chars).
- My Tickets page: list of bookings with PNR, train, date, class, seats, status.
- Cancel a ticket (seats freed, status = CANCELLED).

Admin - Train & Schedule Management

- CRUD for trains (trainNumber, trainName, totalSeatsPerClass).
 - CRUD for schedules (trainId, fromStation, toStation, departureTime, arrivalTime, journeyDate, farePerClass).
-

5) User Flows (Happy Path)

1. Search: User enters From Station, To Station, Date -> sees train list.
 2. Select Class: Clicks a train -> sees class options with fare and availability.
 3. Seat Map: Selects class -> sees seat grid -> picks seats.
 4. Auth: If not logged in -> prompted to register/login.
 5. Confirm: Reviews booking -> confirms -> sees PNR number.
 6. My Tickets: Views all tickets -> can cancel upcoming journeys.
 7. Admin: Logs in -> manages trains and schedules.
-

7) Architecture

- Frontend: React (Hooks + Context API) or Angular (Services + Components). Routing enabled.
 - Backend: Spring Boot (REST Controllers, Services, Spring Data JPA).
 - Database: PostgreSQL (Dev may use H2 for quick start).
 - Auth: Stateless JWT or simple session cookie (choose one; document trade-offs).
 - State: Selected class and seats held in frontend state.
 - Deployment: Localhost
-

8) Tech Stack

Frontend

- React + Vite/CRA (Routing, Hooks, Context) or Angular (Routing, Services)
- HTML5, CSS3, JavaScript (ES6), TypeScript

Backend

- Java 17+, Spring Boot, Spring Web, Spring Data JPA, Validation
- JUnit5, Mockito

Database

- PostgreSQL, JPA/Hibernate
- H2 for local

Tools & Practices

- Git Basics (branching, PRs), Postman/Swagger, Agile ceremonies
- AI Pair Programming: GitHub Copilot/ChatGPT for boilerplate and explanations

9) Data Model

Entities & Relationships

- Train (1) -- (M) Schedule
- User (1) -- (M) Booking
- Schedule (1) -- (M) Booking
- Booking stores: travelClass (ENUM), seatNumbers (CSV string e.g. '1A,1B'), pnrNumber, status

Note: Store booked seats as a CSV string per booking. Derive availability by counting booked seats vs totalSeatsPerClass.

10) API Outline

Auth

- POST /api/auth/register - {email, password, name} -> 201
- POST /api/auth/login -> 200 {token}
- POST /api/auth/logout -> 204

Trains & Schedules

- GET /api/schedules?from=&to=&date;= - search schedules with class availability
- GET /api/schedules/{id}/seats?class= - booked seat numbers for class
- POST /api/schedules (admin)
- PUT /api/schedules/{id} (admin)
- DELETE /api/schedules/{id} (admin)

Trains (Admin)

- GET /api/trains
- POST /api/trains
- PUT /api/trains/{id}
- DELETE /api/trains/{id}

Bookings

- POST /api/bookings - {scheduleId, travelClass, seatNumbers[]} -> creates booking + PNR
- GET /api/bookings/mine - user's tickets
- PUT /api/bookings/{id}/cancel - cancel ticket

Error Format (Consistent)

```
{
  "timestamp": "2025-09-12T12:00:00Z",
  "path": "/api/...",
  "error": "VALIDATION_ERROR",
  "message": "Field is required"
}
```

11) NFRs & SLAs

- Performance:
 - API response ≤ 2 s under light load (1-5 concurrent users locally).
 - UI renders initial content within 3 s on localhost.
- Security:
 - Passwords hashed (BCrypt); JWT/session token stored securely.
 - Basic input validation & server-side checks.
- Scalability/Concurrency:
 - Test with 5-10 concurrent requests locally. Simulate using Postman Runner or multiple browser tabs. No JMeter or load-testing tools required at this level.
- Error Handling:
 - User-friendly error toasts/messages on UI; structured error JSON on API.
- Logging:
 - Console logs; simple request/response logs (no external logging infra).
- Documentation:
 - README.md with setup steps, seed data, and run commands.
 - API contracts documented with Swagger/OpenAPI.

12) Testing

- Backend Unit Tests
 - Services & Repos with JUnit5 + Mockito
 - Target coverage: $\geq 60\%$ (method-level)
- API Testing
 - Manual via Postman or Swagger.
- Frontend
 - Optional unit tests for 1-2 key components (e.g., Booking form).
- No automated regression/e2e required.

13) Engineering PoV

- Code Quality:
 - Follows SOLID basics where practical (e.g., service layer, DTOs).
 - No critical lint errors (ESLint/TSLint).
- Branching:
 - main (stable), feature/* branches; PR reviews by peers/mentor.
- Coverage:
 - Aim 60-70% backend, frontend minimal.

- Dependencies:
 - Keep list short; beware of transitive vulnerabilities.
 - Docs:
 - API endpoints documented; Postman collection included.
-

14) Delivery Plan (3 Sprints, 1-2 weeks each)

Sprint 1 - Foundations & Train Search

- Scaffolding (FE + BE), DB setup, Git workflow
- Train entity, Schedule entity; GET /schedules search endpoint
- Frontend: Home + Search Form + Search Results
- Seed data loading (trains, schedules)

Sprint 2 - Auth & Seat Selection

- Register/Login; protect booking routes
- Class selection UI; seat grid UI; GET booked seats
- Fare calculation; validation + error handling

Sprint 3 - Booking & Admin

- POST /api/bookings with PNR generation; My Tickets page; Cancel ticket
 - Admin: Train CRUD + Schedule CRUD screens
 - Polish: README, Postman collection, unit tests >=60%
-

15) Sample Acceptance Criteria

Train Search

- Given I search Chennai -> Mumbai on a seeded date, I see at least 2 trains.
- Each result shows available classes with seat count and fare.

Seat Selection

- Given I select AC_3 class, I see an 8x8 grid with booked seats highlighted.
- I cannot select an already-booked seat.

Booking

- After confirming, I receive a PNR number on the confirmation screen.
- Cancelling a ticket changes status to CANCELLED and frees the seats.

Admin

- Only admin can access /admin.
 - Admin adds a train and it is selectable when creating a new schedule.
-

16) Example Seed Data (SQL)

SQL

```
INSERT INTO users(id,email,password_hash,is_admin,name,created_at)
VALUES (gen_random_uuid(),'admin@raileasy.com', '$2a$10$hash',true,'Admin',now());
```

```

INSERT INTO train(id,train_number,train_name,seats_per_class,created_at)
VALUES
(gen_random_uuid(),'12163','Chennai Express',64,now()),
(gen_random_uuid(),'22691','Rajdhani Express',64,now());

INSERT INTO schedule(id,train_id,from_station,to_station,departure_time,arrival_time,journey_date,fare_sleeper,fare_ac3,fare_ac2,created_at)
VALUES
(gen_random_uuid(),(SELECT id FROM train WHERE train_number='12163'),'Chennai Central','Mumbai CSMT','2025-10-21 06:00:00','2025-10-22 05:30:00','2025-10-21',450.00,1200.00,1800.00,now()),
(gen_random_uuid(),(SELECT id FROM train WHERE train_number='22691'),'Chennai Central','Mumbai CSMT','2025-10-21 08:00:00','2025-10-22 07:45:00','2025-10-21',500.00,1350.00,2000.00,now());

```

17) Simple Wireframe

- Home: Search panel (From Station / To Station / Date / [Search Trains])
- Search Results: Cards - Train Name + Number / Departure-Arrival / Class buttons (SLEEPER | AC3 | AC2) with fare and seats left
- Seat Grid: 8x8 grid for selected class. Available = green, Booked = red. Total fare + [Confirm Booking]
- My Tickets: Table (PNR / Train / Date / Class / Seats / Status) - [Cancel]
- Admin: Trains table + Schedules table + Add/Edit forms

18) Sample User Stories

- As a passenger, I want to search trains by station and date so I can plan my journey.
- As a passenger, I want to choose a travel class so I can book within my budget.
- As a passenger, I want to see available seats so I can pick my preferred berth.
- As a passenger, I want a PNR number after booking so I can reference my ticket.
- As an admin, I want to add train schedules so passengers can book new routes.

19) Definition of Ready (DoR) & Done (DoD)

DoR

- User story has clear acceptance criteria.
- API/DB changes identified (if any).
- Mock/UX reference exists (even a sketch).

DoD

- Code compiles; unit tests for backend services pass.
- Manual tests completed (Postman + UI).
- No critical ESLint/TS issues; meaningful README update.
- Peer review completed; merged to main.

20) Evaluation Rubric (Aligned to Foundation Skills)

Area	What We Check	Weight
Frontend (React/Angular)	Routing, hooks/services, component communication, state handling (Context/Service), basic TypeScript usage	25%
Backend (Spring Boot)	Clean REST controllers, services, JPA entities & relationships, validation, basic error handling	25%
Common Skills	Core Java correctness, SQL/JPA usage, SOLID basics, Git hygiene, HTML/CSS semantics	20%
Testing	JUnit/Mockito for services/repos; Postman collection; target >=60% coverage	15%
Generative AI Usage	Appropriate use of Copilot/ChatGPT for boilerplate, explanations, refactoring (cite prompts in README)	10%
Documentation & Readme	Setup steps, seed data, API list, constraints & trade-offs	5%

22) Stretch Goals (Optional, If Time Permits)

- Waitlist status when all seats in a class are booked.
- Station autocomplete dropdown.
- Downloadable ticket summary page.

23) Getting Started (Suggested Commands)

Backend

- Java 17+, Spring Boot.
- Profiles: dev (H2), prod (Postgres).

Frontend

- React: `npm create vite@latest -ui -- --template react-ts`
- Angular: `ng new -ui --routing --style=scss`

Run Order

1. Start DB (H2 or Postgres)
 2. Run Spring Boot
 3. Seed Data
 4. Run UI
-